Umpyr Resolve — Technical Architecture

As built on 2026-10-20 (V2 phases 1–3). Companion: "Umpyr Resolve — Business Logic and How It Works".

1. Overview

Umpyr Resolve is an AI dispute-resolution engine for merchants. A customer message about an order or booking comes in; the engine structures it, matches the merchant's record, works out what the merchant's written policy and the governing regulation provide, verifies that against precedent, and either resolves, asks a question, offers a choice, queues for a person, or escalates. Every step is recorded in an append-only, hash-chained ledger, and every policy decision carries a citation back to the document, section, page and clause it rests on.

Design principles, in priority order:

1. **Rules decide, models assist.** Outcomes come from a deterministic rule engine over compiled policy. Language models extract, classify, plan and draft; they never decide and never cite.
2. **Ontology first.** Rules, plans and records reference a fixed field catalogue.
3. **Never assume a fact.** Facts come from the merchant's record or the customer's own words; a choice only the customer can make is presented as options.
4. **Everything cited, everything audited.** Citations are looked up from clause provenance. The ledger is tamper-evident.
5. **PII masked on ingest.** Umpyr holds masked conversation text only.
6. **One small model, budget guarded.** Every runtime call uses the configured Haiku model, is costed, and is attributed to a case.

2. Components

```
browser ──► Cloudflare Pages (demo.umpyr.org)
             │  functions/_middleware.ts: shared-password cookie, /login, /api/* proxy + gateway secret
             └──► Railway: umpyr-engine (FastAPI, Python 3.12, Docker) ──► Railway Postgres (pgvector)
                                                                       ──► Anthropic API (Haiku, daily cap)
```

Component	Technology	Role
Web app	Vite 5, React 18, TypeScript, Tailwind	Customer chat, merchant chat, Ops view, setup/onboarding (hidden in the public build)
Edge	Cloudflare Pages Functions	Password gate, session cookie, API proxy that stamps `x-umpyr-gateway`
Engine	FastAPI, SQLAlchemy 2, Alembic, pydantic-settings	API, resolve runtime, onboarding pipeline, worker
Rules	`packages/rules` (JSON Logic evaluator, PDP, audit hashing)	Deterministic decisions, authorisation, canonical hashing
Ontology	`packages/ontology` (pydantic models, field catalogue, JSON Schemas)	The vocabulary every rule and record uses
Database	Postgres 16 + pgvector	All state; embedded `pgserver` locally, Railway Postgres in production
Models	Anthropic Haiku via the official SDK; local hash embedder (Voyage optional)	Extraction, classification fallback, live planner, drafting, supervision

3. Repository layout

```
apps/web/                React app (src/pages, src/components/OpsView.tsx, src/lib),
Pages functions
packages/ontology/       umpyr_ontology: models, enums, catalog, schema export
packages/rules/          umpyr_rules: model (Clause/Provenance/Rule/RulePack/Ruling),
engine, jsonlogic, pdp, audit
services/engine/         umpyr_engine: api/, resolve/, compile/, learn/, ingest/,
onboard/, sandbox/, llm/, db/, check/
services/engine/migrations/   Alembic (4 revisions)
samples/{rockler,alaska}/ cached synthetic sandboxes: tickets, records, policy archive,
clauses, rule pack, cases, eval
infra/                   Dockerfile, start.sh, deploy.sh, DEPLOY.md
docs/                    design v2, gap list, plan, DECISIONS.md (D1–D54), these
documents
```

4. Data model

Tenant-scoped tables all carry `workspace_id`. Key tables:

Table	Purpose
`workspaces`	Merchant workspace, settings (`as_of`, `record_kind`, `auto_approve_limit`, `approval_sla_hours`)
`tickets`, `orders`	Historical tickets (masked thread, staff actions) and merchant records (order/booking JSON)
`cases`	Mined historical cases: structured record, facts, text embedding, **plan embedding**, drift flags
`categories`, `procedures`	Taxonomy and approved step procedures per category
`policy_documents`, `clauses`, `rule_packs`, `rules`	Compiled policy with per-clause provenance (document version hash, section, page, span, text hash, public flag)
`drift_findings`, `router_models`, `eval_scenarios`, `next_questions`	Learning artefacts
`chat_sessions`, `chat_messages`	Conversations (masked) with the per-turn Ops snapshot
`resolve_cases`	One case per conversation: status, state, typed escalation, cost, ledger head and decision hashes, `due_at`
`case_events`	Append-only hash chain, trigger-protected
`rulings`	Per-turn audit record with canonical SHA-256
`llm_calls`, `daily_spend`, `embedding_calls`	Cost per call, per case, per day

Ledger event hash: `sha256(canonical({case_id, seq, prev_hash, turn, actor, action, basis, result, cost_usd, at}))`. Decision commitment: `sha256(head_hash ‖ ruling_audit_hash)`. A Postgres trigger refuses UPDATE and DELETE on `case_events` (retention cascade excepted). `GET /cases/{id}/verify` recomputes the chain.

5. Onboarding pipeline (offline, per merchant)

```
fetch policy sources ─► compile (segment → clauses → JSON Logic rules → overrides → pack)
ingest tickets + records (PII masked) ─► mine structured cases ─► taxonomy (clusters, categories)
─► router (embedding + logistic regression) ─► procedures (common staff path + clauses)
─► drift (policy vs practice replay) ─► provenance (locate clauses in documents)
─► plan embeddings (structured case text) ─► next-question model (follow-up patterns)
```

- **Compiler.** Documents are chunked; Haiku extracts verbatim clauses, then converts each into a rule with `tier` (law > contract > commitment > goodwill), `priority`, scope (`categories`, `claim_reasons`), `requires` (fact paths), `when` (JSON Logic) and `effect` (decision, remedy, amount expression). Merchant overrides (`rule_overrides.json`: enable_only / disable / patch / add) model the merchant approval step; reassembly needs no model calls.
- **Provenance.** Each clause is located in the archived document text (exact, whitespace-insensitive, then first-words anchor, curly quotes folded); the nearest heading, the PDF page and the character span are stored with a hash of the clause text and the document's content hash. In the sandboxes 597 of 600 clauses are located.

- **Sandboxes.** Rockler and Alaska Airlines use real public policies and synthetic tickets; `umpyr sandbox seed` loads everything from cache with the LLM cap forced to zero, and backfills provenance, plan embeddings, next questions, SLA settings and a newer cached rule pack at boot.

6. Resolve runtime (per customer message)

`resolve/runtime.py` is a code-driven state machine. One message = one turn inside the case ledger and a cost meter:

#	Stage	What happens	Ledger event
0	Receive	PII masked; case opened or loaded; Check ingestion hook (no-op)	`message.received`, `check.ingest`
0b	Small talk	A message with no ask and no new fact gets a one-line acknowledgement; no model call	`acknowledgement`
1	Classify	Router over the whole conversation; Haiku fallback below 0.70; the latest message classified alone on later turns	`classification`
2	Precedent	k=3 text neighbours (display)	`precedent.retrieved`
3	Procedure	Approved procedure for the category	`plan.selected`
4	Record	Lookup by reference; unknown references get a stand-in record cloned from the nearest case, aligned to the claim and scrubbed of template-only facts (D37, D52)	`record.lookup`
5	Extract	Schema v2 structured extraction (intents, party, timeline ask, claim, airline facts); delta mode from turn 2; facts derived from evidence (matched record = proof of purchase, bag facts, cause grounded in the customer's words); customer choice detection	`extraction`, `choice`
5b	Plan	Historical consensus plan vs live Haiku plan, compared by step type and order; novelty; confidence	`plan.historical`, `plan.live`, `plan.compared`
6	Rule	Deterministic evaluation; options and choices; third-ask guard	`rule_check`, `options_offered`, `choices_offered`
7	Gate	PDP authorisation (role, licence, amount, mode, limit); Check pre-execution hook; planner gate; typed escalation; uncovered-ask precedent; queue with SLA	`check.pre_execution`, `gate`, `escalation`, `precedent.goodwill`, `queue.overdue`, `queue.withdrawn`
7c	Prefetch	Predicted next questions; cited refund-timeline answer	`prefetch`
8	Draft	Haiku drafts the reply from the ruling payload; the supervisor checks it against the ruling and the customer's message; one regeneration, then a humane template	`notification.drafted`
9	Audit	Per-turn ruling record with canonical hash; decision hash committed on the chain	`decision`

Statuses: `resolved`, `denied`, `needs_fact`, `needs_choice`, `pending_approval`, `escalated`.

7. Rule engine and precedence

`umpyr_rules.evaluate(pack, facts, category)`:

1. Rules are sorted by `(tier rank, -priority, rule_id)` and filtered by scope.
2. A rule whose required facts are missing is undecidable; the first undecidable level is remembered.
3. Rules whose `when` evaluates true fire; the first fired rule wins; a different decision or remedy at the same level is a conflict → escalate.
4. **D53:** if the undecidable level outranks the winning fired rule, the ruling is `needs_fact` for those facts. A commitment-tier credit can never pre-empt an undecided regulation.
5. With nothing fired: `needs_fact` if facts are missing, otherwise `escalate` ("no compiled rule matched").

The PDP (`authorize`) checks actor role, structural actions, whether the ruling licenses the action, amount bounds (≤ ruled amount, ≤ record value), session mode (autonomous / assist / shadow), rule-required approval and the auto-approve limit.

8. Planner (design v2 §4)

- **Structured case text**: fixed-order serialisation (category, intents, claim reason, requested remedy, disruption, bag, item condition, party). Historical cases are embedded on `cases.plan_embedding`.
- **Historical plan**: k=5 nearest cases; their staff-action sequences are normalised (verification steps → `verify`, evidence requests → `evidence`, remedy verbs kept verbatim) and aggregated into a consensus with support, agreement and mean similarity. History is *confident* with ≥3 neighbours, agreement ≥0.5 and the consensus remedy supported by ≥60%.
- **Live plan**: Haiku decomposes the intents into steps from the fixed vocabulary.
- **Comparison**: `agree`, `order_differs`, `minor_diff` (only verification/evidence differ), `diverge` (different remedy steps), `no_live`, `no_history`.
- **Gate**: a divergence queues an otherwise auto-approved money, shipment or cancellation action only when history is confident **and** the ruling's remedy contradicts that precedent (`planner_disagreement`). The live planner alone never gates; a rebooking is never gated.

9. Customer-experience layer (`resolve/customer.py`)

- Status strip from templates; public-document citations only; refund options from fired rules (cash first where regulation guarantees it); choice branches evaluated per `CHOICE_FACTS`; next-question model (`next_questions`) and a cited refund-timeline answer; approval queue with per-workspace SLA and honest overdue escalation; goodwill precedent for uncovered asks; small-talk detection (pleasantry vocabulary only).

10. LLM layer

- `LLMProvider` protocol (`complete`, `batch`); `ClaudeProvider` enforces a Haiku model id, structured outputs (`output_config` JSON schema; arrays must not use `maxItems`), prompt caching, and records every call to `llm_calls` with cost, latency, `case_id` and `session_id`.

- `BudgetGuard` blocks at `DAILY_COST_CAP_USD` per UTC day (HTTP 402 → "Demo budget reached for today"). Public demo cap: $1/day.
- Per-turn cost: about $0.001–0.003 (6–10 calls: extract, plan, draft, supervise, classify fallback, regenerate). A small-talk turn costs nothing.
- Embeddings: local hashing embedder by default (free, deterministic); Voyage when a key is set. Novelty thresholds must be recalibrated when switching.

11. API surface

Route	Purpose
`POST /workspaces/{id}/sessions`, `POST /sessions/{id}/messages`	Start a chat, send one turn → `{reply, ops, customer, session}`
`GET /sessions/{id}/case`, `GET /cases/{id}`, `/events`, `/verify`	Case object, ledger, chain verification
`GET /workspaces/{id}/metrics`	Cases by status, escalations by type, cost per resolved case, time to resolution
`GET /workspaces/{id}/audit`, `/rules`, `/procedures`, `/drift`, `/examples`, `/summary`	Ops and setup data
`POST /workspaces/{id}/merchant-chat`	Merchant questions answered from computed aggregates
`POST /workspaces`, uploads, `POST /onboard`, `GET /readiness`, `PATCH` approvals	Upload-and-onboard flow (hidden in the public build)
`GET /health`, `GET /admin/spend`	Health and spend

All routes except `/health` require `x-umpyr-gateway` when `API_GATEWAY_SECRET` is set; only the Pages middleware adds it.

12. Web app

Routes: `/` (workspaces, health, spend), `/:client` (customer chat, merchant chat, Ops view), plus `/try`, `/:client/onboard`, `/:client/setup` compiled out of the public build. The Ops view shows, per turn: "This turn" (message category, new asks, facts changed, ruling changed, precedent), classification, neighbours, plan (history vs live), procedure with rule chips linking to document/section/page, ruling with citations and escalation type, governance (policy version, authority, supervisor, audit hash, ledger hashes, case cost), customer status timeline, tool trace. The chat shows the status strip, choice and option buttons, SLA line and source links.

13. Deployment and security

- **Railway**: Dockerfile build (`RAILWAY_DOCKERFILE_PATH`), `start.sh` runs migrations, the idempotent seed/backfill, then the API. Variables: model, API key, cap, gateway secret, CORS, `ASSUME_REFERENCE_EXISTS`, `PLANNER_GATE`.
- **Cloudflare Pages**: static build + functions; secrets `DEMO_PASSWORD`, `SESSION_SECRET`, `API_GATEWAY_SECRET`, `API_ORIGIN`; custom domain `demo.umpyr.org` (CNAME, proxied); `noindex` and `robots.txt` disallow.

- **Releases**: `make deploy` (tokens from `.env`, never printed); `infra/deploy.sh` subcommands `bootstrap`, `engine`, `web`, `secrets`. Git history is scanned for credential shapes before every push.
- **Data**: synthetic only; PII masked before storage and before any model call; `.env` never read by the agent, never committed.

14. Testing

139 engine tests (pytest against an ephemeral embedded Postgres, fake LLM and embedder; the user's real conversations are regression tests), 44 rules tests, 5 ontology tests, web typecheck and vitest. Real-model checks are run manually against the local sandboxes at a few cents each.

15. Configuration (selected)

`CLAUDE_MODEL`, `DAILY_COST_CAP_USD`, `LLM_PROVIDER`, `EMBEDDING_PROVIDER`, `DATABASE_URL`, `API_GATEWAY_SECRET`, `CORS_ORIGINS`, `ASSUME_REFERENCE_EXISTS`, `PLAN_NEIGHBOURS_K`, `NOVELTY_MIN_SIMILARITY`, `PLAN_AGREEMENT_MIN`, `PLANNER_GATE`, `PROACTIVE_MIN`, `CHECK_PROVIDER`; workspace settings `auto_approve_limit`, `approval_sla_hours`, `as_of`, `record_kind`.

16. Known limitations and roadmap

- No execution layer yet: remedies are decided and queued, not executed against a merchant system (phase 4: operations ontology, bindings, idempotent execution, system-of-record confirmation, hard gate with duplicate-refund and cap checks, approval queue with decision cards, settlement-time model).
- No feedback loop yet: live cases and human decisions do not feed the precedent store or rule candidates (phase 5: labels from the queue, scorecards, graduation, policy-document drift, eval scoreboard).
- Single-winner rule semantics: a case that deserves two remedies (rebooking plus expense reimbursement) resolves one and names the other for a specialist.
- Stand-in records for unknown references are a POC convenience (`ASSUME_REFERENCE_EXISTS`); in production the merchant's system of record is authoritative.
- The local hash embedder makes neighbour similarity nearly uniform; novelty rests on agreement until Voyage is enabled.
- Interim customer status is reported after the turn (one synchronous call), not streamed.

Appendix: decision index

D1–D35 V1 (precedence, PDP, fixed state machine, compiler, needs_fact focus, drift semantics); D36–D37 references; D38–D39 deployment; D40 design v2 as source of truth; D41 provenance and citations; D42 case ledger; D43 extraction v2 and delta; D44 planners and gate; D45 novelty; D46 status strip; D47 public citations; D48 next questions and timeline; D49 options, choice, queue SLA; D50 derived bag facts and the third-ask guard; D51 uncovered asks and goodwill precedent; D52 never assume a choice or invent a fact; D53 undecided higher rule blocks; D54 grounded facts and supervisor context. Full text in `docs/DECISIONS.md`.